

PROGRAM 11:-

AIM

To implement the Map Colouring Problem using Constraint Satisfaction Problem (CSP) by assigning colours to different regions such that no two adjacent regions have the same colour.

ALGORITHM

1. Start the program.
2. Define the regions and their neighbouring regions.
3. Define a set of available colours.
4. Select an uncoloured region.
5. Assign a colour to the selected region.
6. Check whether the assigned colour is different from the colours of all adjacent regions.
7. If the assignment is valid, move to the next region.
8. If the assignment is invalid, try another colour.
9. Continue until all regions are coloured.
10. Display the valid colour assignment.
11. Stop the program.

PROGRAM

```
def is_valid(region, color, assignment, graph):  
    for neighbour in graph[region]:  
        if assignment.get(neighbour) == color:  
            return False  
    return True  
  
def map_coloring(graph, colors, assignment):  
    if len(assignment) == len(graph):  
        return True  
    region = next(r for r in graph if r not in assignment)  
    for color in colors:  
        if is_valid(region, color, assignment, graph):  
            assignment[region] = color
```

```

        if map_coloring(graph, colors, assignment):
            return True

        del assignment[region]

    return False

graph = {
    'A': ['B', 'C'],
    'B': ['A', 'C', 'D'],
    'C': ['A', 'B', 'D'],
    'D': ['B', 'C']
}

colors = ['Red', 'Green', 'Blue']

assignment = {}

if map_coloring(graph, colors, assignment):
    print("Map Coloring Solution:")
    for region, color in assignment.items():
        print(region, "->", color)
else:
    print("No solution exists")

```

OUTPUT

```

Map Coloring Solution:
A -> Red
B -> Green
C -> Blue
D -> Red

```

RESULT

The Map Colouring Problem was successfully implemented using the CSP technique, and valid colours were assigned to all regions such that adjacent regions have different colours